

E-Commerce Customer & Sales Intelligence
SQL Data Analytics Project Documentation

Technology: MySQL 8.0 | SQL | MySQL Workbench | MySQL Shell

Prepared By: Divesh Sonawane
Degree: B.E. Computer Engineering

Project Domain: Data Analytics / Business Intelligence / E-Commerce
Project Type: Portfolio / Industry-Oriented SQL Analytics Project
Project Status: Completed — SQL Analytics Version

Year: 2026

Table of Contents

1. Project Overview
 2. Business Problem
 3. Project Objectives
 4. Dataset Overview
 5. Tools & Technologies
 6. Database Architecture
 7. Data Import Process
 8. Data Validation & Business Analysis
 9. Advanced SQL Techniques and Analysis
 10. Advanced Analysis Examples
 11. Business Insights
 12. Project Challenges & Solutions
 13. SQL File Documentation
 14. Project Limitations
 15. Future Enhancements
 16. Conclusion
 17. GitHub Repository
-

1. Project Overview

Project Overview

E-Commerce Customer & Sales Intelligence is an SQL-based data analytics project developed using the Brazilian Olist E-Commerce dataset to transform raw transactional data into meaningful business insights and Key Performance Indicators (KPIs). The project integrates customer, order, order-item, product, and payment data within a relational MySQL 8.0 database, enabling comprehensive analysis of e-commerce sales performance, customer purchasing behavior, product performance, payment patterns, order status, and operational trends.

The project follows a structured data analytics workflow, starting with database design, CSV data ingestion, data validation, data quality assessment, NULL-value analysis, duplicate detection, and referential integrity checks, followed by exploratory and business-focused SQL analysis. More than 99,000 orders, 112,000+ order-item records, 103,000+ payment records, and 96,000+ unique customers were analyzed using SQL to derive actionable insights.

Advanced SQL techniques including INNER JOIN, LEFT JOIN, GROUP BY, HAVING, CASE statements, aggregate functions, Common Table Expressions (CTEs), subqueries, window functions, RANK(), ROW_NUMBER(), LAG(), running totals, time-series analysis, Month-over-Month (MoM) growth, customer segmentation, and revenue contribution analysis were implemented to solve practical business questions.

The project focuses on key Data Analyst and Business Analyst use cases, including revenue analysis, Average Order Value (AOV), customer retention and repeat-purchase analysis, product and category performance, payment analysis, cancellation-rate analysis, geographic analysis, delivery performance, customer-level revenue ranking, and sales trend analysis. The project also demonstrates the ability to identify and quantify real-world data-quality issues, such as missing values and orphan product references, rather than assuming the source data is perfectly clean.

Overall, the project demonstrates practical proficiency in SQL, MySQL, relational database concepts, data cleaning, data validation, exploratory data analysis, business intelligence, KPI development, analytical problem-solving, and translating business requirements into data-driven insights. It provides a foundation for further development into Business Intelligence, Data Engineering, ETL pipelines, data warehousing, dashboarding, and advanced customer analytics.

2. Business Problem

Problem Statement

E-commerce businesses generate large volumes of data across multiple operational systems. Customer information, orders, products, payments, and item-level transactions are often stored separately.

The business needs a structured analytical approach to answer questions such as:

- How many orders are being generated?
- How much revenue is being generated?
- Which products generate the most revenue?
- Which customers contribute the most revenue?
- How many customers make repeat purchases?
- What is the repeat-customer rate?
- What is the average order value?
- Which payment methods are most commonly used?
- How frequently are orders canceled?
- How is sales performance changing over time?
- Which locations contribute most to sales?
- How effectively are orders being delivered?

The project addresses these questions through SQL-based analysis.

3. Project Objectives

Primary Objectives

1. Build a relational MySQL database from raw e-commerce datasets.
 2. Import and validate large CSV datasets.
 3. Establish relationships between business entities.
 4. Perform data-quality checks.
 5. Analyze sales and revenue performance.
 6. Analyze customer purchasing behavior.
 7. Identify repeat and one-time customers.
 8. Analyze product performance.
 9. Analyze payment behavior.
 10. Analyze order-status distribution.
 11. Analyze delivery performance.
 12. Perform time-series revenue analysis.
 13. Apply advanced SQL techniques.
 14. Generate business-oriented insights from transactional data.
-

4. Dataset Overview

4.1 Dataset Name

Brazilian E-Commerce Public Dataset by Olist

The project uses the Olist Brazilian E-Commerce Public Dataset, a historical e-commerce dataset containing information related to customers, orders, products, sellers, order items, payments, and delivery operations.

For this project, five core datasets were selected and loaded into a MySQL 8.0 relational database:

1. olist_customers_dataset.csv
2. olist_orders_dataset.csv
3. olist_order_items_dataset.csv
4. olist_order_payments_dataset.csv
5. olist_products_dataset.csv

These datasets were selected because they provide the necessary relationships to perform customer, sales, product, payment, and operational analysis.

4.2 Dataset Purpose

The dataset provides a realistic transactional environment for analyzing an e-commerce business.

The data can be used to investigate:

- Customer purchasing behavior
 - Order volume and order status
 - Product performance
 - Sales and revenue
 - Freight and shipping costs
 - Payment methods
 - Installment behavior
 - Customer retention and repeat purchases
 - Delivery performance
 - Geographic sales distribution
 - Monthly and seasonal sales trends
 - Data-quality and referential-integrity issues
-

4.3 Dataset Summary

Dataset	Database Table	Records Used	Primary Analytical Purpose
Customers	customers	99,441	Customer and geographic analysis
Orders	orders	99,441	Order and lifecycle analysis
Order Items	order_items	112,650	Transaction and revenue analysis
Order Payments	order_payments	103,886	Payment and transaction analysis
Products	products	32,340	Product and category analysis

Total records across the five tables: approximately 447,857 records.

Note: The tables contain different levels of granularity. Therefore, the total record count should not be interpreted as the number of unique business transactions or customers.

4.4 Customer Dataset

File: olist_customers_dataset.csv

Database Table: customers

Records: 99,441

The customer dataset contains customer identifiers and geographic information. It provides the customer dimension required to connect customers with their orders and perform location-based and customer-level analysis.

Columns

Column	Data Type	Description
customer_id	VARCHAR	Unique customer/order relationship identifier
customer_unique_id	VARCHAR	Identifier representing the unique customer
customer_zip_code_prefix	INT	Customer ZIP-code prefix
customer_city	VARCHAR	Customer city
customer_state	VARCHAR	Customer state

Analytical Uses

- Unique customer identification
- Repeat-customer analysis
- One-time vs repeat customers

- Customer revenue analysis
- Customer order frequency
- State-level customer distribution
- Geographic analysis

4.5 Orders Dataset

File: olist_orders_dataset.csv

Database Table: orders

Records: 99,441

The orders dataset represents the order-level transaction lifecycle. It contains information about customers, order status, purchase timestamps, approval, shipping, delivery, and estimated delivery dates.

Columns

Column	Data Type	Description
order_id	VARCHAR	Unique order identifier
customer_id	VARCHAR	Customer associated with the order
order_status	VARCHAR	Current status of the order
order_purchase_timestamp	DATETIME	Date and time when the order was placed
order_approved_at	DATETIME	Date and time when payment/order approval occurred
order_delivered_carrier_date	DATETIME	Date when the order was handed to the carrier
order_delivered_customer_date	DATETIME	Date when the order reached the customer
order_estimated_delivery_date	DATETIME	Estimated customer delivery date

Analytical Uses

- Total order analysis
- Order-status analysis
- Cancellation analysis
- Monthly order trends
- Delivery performance
- Purchase-to-delivery analysis
- Month-over-Month growth
- Order lifecycle analysis

4.6 Order Items Dataset

File: olist_order_items_dataset.csv

Database Table: order_items

Records: 112,650

The order-items dataset contains item-level transactional information. An individual order can contain multiple products, making this table more granular than the orders table.

Columns

Column	Data Type	Description
order_id	VARCHAR	Associated order identifier
order_item_id	INT	Sequential item number within an order
product_id	VARCHAR	Product identifier
seller_id	VARCHAR	Seller identifier
shipping_limit_date	DATETIME	Seller shipping deadline
price	DECIMAL	Price of the item
freight_value	DECIMAL	Freight/shipping cost

Analytical Uses

- Item-level revenue
- Product performance
- Product ranking
- Freight analysis
- Order value calculation
- Seller-level analysis
- Revenue contribution
- Product-category performance

4.7 Order Payments Dataset

File: olist_order_payments_dataset.csv

Database Table: order_payments

Records: 103,886

The payment dataset contains payment information associated with orders. It includes payment methods, payment amounts, payment sequence, and installment information.

Columns

Column	Data Type	Description
order_id	VARCHAR	Associated order identifier
payment_sequential	INT	Sequence number of payment
payment_type	VARCHAR	Payment method used
payment_installments	INT	Number of installments
payment_value	DECIMAL	Payment amount

Analytical Uses

- Payment-method analysis
- Payment-value analysis
- Installment analysis
- Average payment value
- Payment frequency
- Order-level payment analysis
- Customer payment behavior

4.8 Products Dataset

File: olist_products_dataset.csv

Database Table: products

Records Used: 32,340

The products dataset contains product-level attributes including category, description characteristics, photographs, weight, and dimensions.

Columns

Column	Data Type	Description
product_id	VARCHAR	Unique product identifier
product_category_name	VARCHAR	Product category
product_name_lenght	INT	Length of product name
product_description_lenght	INT	Length of product description
product_photos_qty	INT	Number of product photographs
product_weight_g	DECIMAL	Product weight in grams

Column	Data Type	Description
product_length_cm	DECIMAL	Product length
product_height_cm	DECIMAL	Product height
product_width_cm	DECIMAL	Product width

Analytical Uses

- Product performance
- Category analysis
- Revenue by product
- Revenue by category
- Product ranking
- Product attribute analysis
- Missing-product relationship analysis

4.9 Dataset Relationships

Primary Relationships

Parent Table	Child Table	Relationship	Key
Customers	Orders	One-to-Many	customer_id
Orders	Order Items	One-to-Many	order_id
Orders	Order Payments	One-to-Many	order_id
Products	Order Items	One-to-Many	product_id

4.10 Data Granularity

Understanding granularity was important when performing the analysis.

Table	Granularity
Customers	One customer/order relationship record
Orders	One record per order
Order Items	One record per product item within an order
Payments	One record per payment associated with an order
Products	One record per product

Because these tables have different granularities, direct aggregation across multiple one-to-many tables can cause double counting if joins are not designed carefully.

Therefore, the project uses appropriate aggregation and intermediate queries/CTEs when calculating metrics such as revenue, payment value, and order-level KPIs.

The dataset provides a relational representation of an e-commerce ecosystem containing customers, orders, products, transactions, payments, and logistics information. With more than 99,000 orders and 100,000+ item and payment records, it provides sufficient scale for demonstrating practical SQL querying and analytical techniques.

The dataset therefore serves as the foundation for the project's subsequent data-quality assessment, exploratory analysis, customer analytics, sales analysis, product analysis, payment analysis, operational analysis, and advanced SQL analysis.

5. Tools & Technologies

Database

- MySQL 8.0.46

Database Tools

- MySQL Workbench
- MySQL Shell

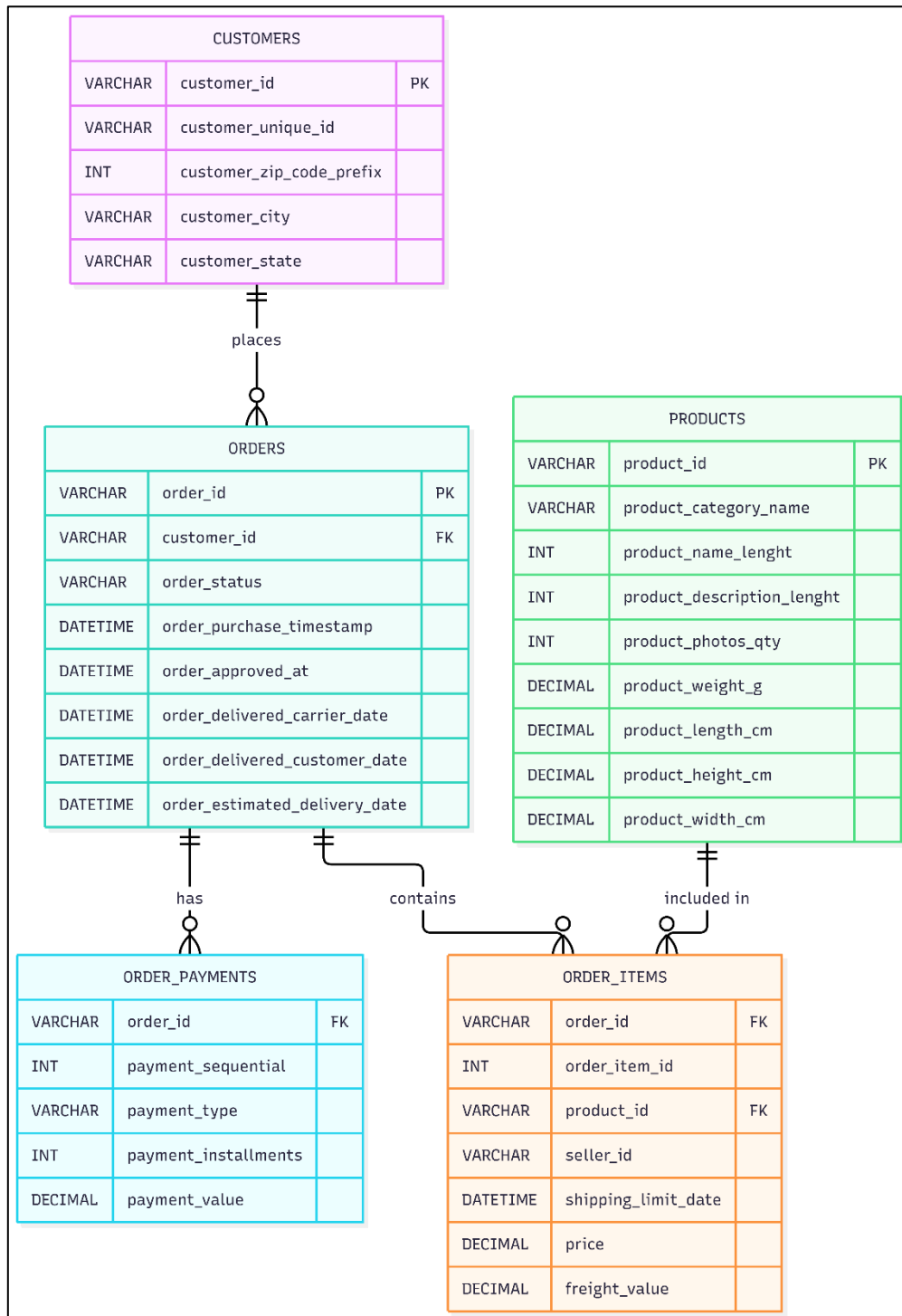
Development

- SQL
- Git
- GitHub

Optional Supporting Tools

- Python
 - CSV
-

6. Database Architecture



Relationship meaning

Relationship	Meaning
Customers → Orders	One customer can have multiple orders
Orders → Order Items	One order can contain multiple items
Orders → Payments	One order can have one or multiple payment records
Products → Order Items	One product can appear in multiple order items

Database Architecture Explanation

The project follows a relational database architecture implemented in MySQL 8.0, consisting of five interconnected tables: customers, orders, order_items, order_payments, and products. The customers table acts as the customer dimension and connects to the orders table through customer_id, allowing customer-level order analysis. The orders table serves as the central order-level entity and connects to order_items and order_payments through order_id, enabling detailed analysis of products purchased, revenue, freight, and payment behavior. The order_items table further connects to the products table through product_id, allowing product and category-level analysis. This one-to-many relational structure supports efficient SQL JOINS, aggregations, CTEs, window functions, KPI calculations, customer segmentation, revenue analysis, and data-quality validation, while maintaining clear separation between customer, order, transaction, payment, and product-level data.

7. Data Import Process

Process

1. Installed/configured MySQL 8.0.
2. Created ecommerce_analytics database.
3. Created relational tables.
4. Imported CSV files.
5. Used MySQL Shell and Python scripts where required.
6. Validated record counts.
7. Performed duplicate and NULL checks.
8. Performed relationship/orphan checks.

Database

```
CREATE DATABASE ecommerce_analytics;
```

Data Import Process

The dataset was imported into the MySQL 8.0 database in two stages. First, the customers and products datasets were imported using the MySQL Workbench Table Data Import Wizard, which provided a straightforward way to map CSV columns to the corresponding database fields and validate the imported records. Due to the larger size of the remaining transactional datasets, the orders, order_items, and order_payments files were imported using Python-based data-loading scripts with optimized CSV processing. This approach provided a more reliable and efficient method for handling larger files while maintaining the required data types and table structure. After each import, record counts and data-quality checks were performed to verify that the data was loaded correctly into the MySQL database.

8. Data Validation and Business Analysis

After completing the data import process, the datasets were subjected to systematic data validation, data-quality assessment, financial validation, relational integrity checks, and business analysis using MySQL. The objective was to ensure that the imported data was structurally reliable and suitable for analytical queries while identifying any limitations present in the original source data.

8.1 Duplicate Data Validation

Duplicate checks were performed on the major identifying fields and business keys across the database.

The following validations were performed:

- Duplicate customer_id
- Duplicate customer_unique_id
- Duplicate order_id
- Duplicate product_id
- Duplicate (order_id, order_item_id) combinations
- Duplicate payment records based on the relevant payment identifiers

The duplicate checks returned 0 problematic duplicates for the tested key fields.

This indicates that the primary identifiers and tested business-key combinations were sufficiently unique for the subsequent analysis.

8.2 NULL Value Analysis

NULL-value analysis was performed to identify missing information across important fields, particularly the order lifecycle timestamps.

Column	NULL Records
order_id	0
customer_id	0
order_status	0
order_purchase_timestamp	0
order_approved_at	160
order_delivered_carrier_date	1,783
order_delivered_customer_date	2,965
order_estimated_delivery_date	0

Interpretation

The missing values were concentrated in order approval and delivery-related timestamps. These values were not automatically treated as data errors because an order may not progress through every stage of the fulfillment lifecycle.

For example, canceled, unavailable, processing, or otherwise incomplete orders may not have a carrier-delivery or customer-delivery timestamp.

Therefore, NULL values were interpreted according to the operational meaning of each field rather than being blindly removed.

8.3 Referential Integrity Analysis

Referential-integrity checks were performed to identify records in child tables that did not have corresponding records in their parent tables.

Relationship	Result
Order → Customer orphan records	0
Order Item → Order orphan records	0
Order Item → Product orphan records	1,604

The order-item analysis identified 1,604 records referencing products that were not present in the imported products table.

These records corresponded to 611 unique missing product IDs.

Financial Impact of Missing Product References

Metric	Value
Affected item revenue	181,469.28
Affected freight value	28,196.81

Interpretation

This represents a source-data referential-integrity limitation. The order-item transactions exist, but their corresponding product dimension records are missing from the imported product dataset.

Instead of deleting these transactions, the project retained them and documented the limitation so that the affected revenue would not be silently excluded from the analysis.

8.4 Financial Data Validation

Financial fields were checked using minimum, maximum, and average-value calculations to identify unusual or potentially invalid values.

Order Item Financial Metrics

Metric	Value
Minimum Item Price	0.85
Maximum Item Price	6,735.00
Average Item Price	120.65
Minimum Freight Value	0.00
Maximum Freight Value	409.68

Payment Financial Metrics

Metric	Value
Minimum Payment Value	0.00
Maximum Payment Value	13,664.08
Average Payment Value	154.10

These checks helped validate the numerical ranges of the transactional and payment data before performing revenue and customer analysis.

8.5 Order Status Analysis

The order_status field was analyzed to understand the overall order lifecycle and distribution of orders across different operational stages.

Order Status	Orders
Delivered	96,478
Shipped	1,107
Canceled	625
Unavailable	609
Invoiced	314
Processing	301
Created	5
Approved	2
Total	99,441

The analysis shows that the majority of orders in the dataset reached the Delivered status, while the remaining orders were distributed across various pre-delivery and exception states.

8.6 Cancellation Analysis

Cancellation performance was evaluated using the total number of canceled orders compared with the total order volume.

- Canceled Orders: 625
- Total Orders: 99,441
- Cancellation Rate = (Canceled Orders / Total Orders) $\times$ 100
- Cancellation Rate $\approx$ 0.63%

This metric provides a high-level measure of the proportion of orders recorded with a canceled status within the dataset.

8.7 Revenue Analysis

Revenue analysis was performed using item-level transactional data from the `order_items` table.

Metric	Value
Total Item Revenue	13,591,643.70
Total Freight Value	2,251,909.54

Item revenue was calculated using the price field from the order-item transactions, while freight was calculated separately using the `freight_value` field.

The analysis keeps item revenue and freight value separate to avoid incorrectly treating shipping charges as product revenue.

8.8 Average Order Value (AOV)

The project calculated the Average Order Value (AOV) using order-level item revenue.

- Formula
$$\text{AOV} = \text{Total Order-Level Item Revenue} / \text{Number of Orders}$$
- Result
$$\text{Average Order Value} = 137.75$$

The AOV provides an overall indication of the average value of item purchases associated with an order in the analyzed dataset.

8.9 Monthly Revenue Analysis

Monthly revenue analysis was performed by extracting the year and month from `order_purchase_timestamp` and aggregating order-item revenue.

- The analyzed period spans: September 2016 → September 2018
- The analysis identified substantial growth in transaction activity during 2017 and 2018.
- The cumulative item revenue calculated across the analyzed period reached: 13,591,643.70

Monthly aggregation was used as the foundation for subsequent time-series analysis and Month-over-Month growth calculations.

8.10 Month-over-Month Revenue Analysis

Month-over-Month (MoM) revenue growth was calculated using the SQL `LAG()` window function to compare each month's revenue with the previous month's revenue.

Formula

MoM Growth % = ((Current Month Revenue – Previous Month Revenue) / Previous Month Revenue) × 100

Examples

Month	MoM Growth
2017-11	+52.10%
2017-12	–26.36%
2018-01	+27.71%
2018-03	+16.47%
2018-06	–13.19%

Interpretation

The monthly analysis demonstrates substantial variation in revenue growth. Some months experienced strong increases compared with the preceding month, while other periods experienced revenue contraction.

The analysis demonstrates practical use of window functions, time-series analysis, `LAG()`, calculated fields, and business KPI analysis.

8.11 Customer Analysis

Customer behavior was analyzed using `customer_unique_id` rather than simply counting `customer_id` records.

Results

Customer Metric	Count
Unique Customers	96,096
Repeat Customers	2,997
One-Time Customers	93,099
Repeat Customer Rate	3.12%

Repeat Customer Definition

A customer was classified as a repeat customer when the same `customer_unique_id` was associated with more than one order.

- Repeat Customer Rate = $(\text{Repeat Customers} / \text{Unique Customers}) \times 100$
- Repeat Customer Rate $\approx 3.12\%$

This distinction between `customer_id` and `customer_unique_id` was important because a single real-world customer can be associated with multiple customer records.

8.12 Customer Segmentation

Customers were segmented according to their order frequency.

Segmentation Logic

- One-Time Customer $\rightarrow$ 1 order
- Repeat Customer $\rightarrow$ More than 1 order

This segmentation provides a simple behavioral classification that can support:

- Customer retention analysis
- Repeat-purchase analysis
- Customer loyalty analysis
- Customer engagement strategies
- Identification of high-frequency customers

The segmentation was implemented using SQL aggregation and conditional logic.

8.13 Top Customers by Revenue

Customer-level revenue analysis was performed by joining customer, order, and order-item data and aggregating transaction values at the customer level.

The analysis used:

- `SUM(price)` for revenue calculation
- `COUNT(DISTINCT order_id)` for order frequency

- GROUP BY for customer-level aggregation
- ORDER BY for revenue ranking
- SQL ranking/window functions for identifying high-value customers

This analysis helped identify customers contributing significant revenue and demonstrated the practical application of customer-level aggregation and ranking techniques.

8.14 Product Analysis

Product performance was analyzed by connecting the products table with order_items through product_id.

The analysis covered:

- Product revenue
- Number of orders
- Product sales frequency
- Product contribution to revenue
- Product/category performance
- Missing product references

The following SQL techniques were applied:

- JOIN
- GROUP BY
- SUM()
- COUNT()
- RANK()
- ROW_NUMBER()
- ORDER BY

The analysis also incorporated the previously identified 1,604 order-item records associated with missing product references, ensuring that the data-quality limitation was considered when interpreting product-level results.

8.15 Order-Item Analysis

The order_items table represents the most granular transactional level used in the project.

Dataset Size: 112,650 order-item records

The analysis included:

- Item price
- Freight value
- Item-level revenue
- Order frequency
- Product contribution
- Revenue ranking

- Product-level transaction volume

Because an individual order can contain multiple items, the `order_items` table was treated as a separate transaction-level grain rather than being directly interpreted as one row per order.

This distinction helped prevent incorrect aggregations and double counting.

8.16 Payment Analysis

The payment dataset contained: 103,886 payment records

Payment behavior was analyzed using:

- Payment type
- Payment value
- Payment installments
- Payment sequence
- Order-level payment behavior
- Payment frequency

The analysis used SQL aggregation and grouping to understand how payments were distributed across orders and payment methods.

Key SQL concepts included:

GROUP BY, COUNT(), SUM(), AVG(), CASE, and JOIN operations.

8.17 Geographic Analysis

Customer geographic information was analyzed using the `customers` table and its relationship with the `orders` and `transaction` tables.

The analysis can be used to evaluate:

- Customer distribution by state
- Order volume by state
- Revenue contribution by state
- Customer concentration
- Geographic purchasing patterns

This analysis demonstrates the ability to combine dimension data with transactional data using relational SQL joins.

The geographic analysis also demonstrates a practical business-intelligence use case, where customer location can be combined with transaction metrics to understand regional performance.

8.18 Overall Data Validation and Analysis Outcome

The validation and analytical process established that the database was sufficiently structured for SQL-based business analysis while also identifying important source-data limitations. The project successfully validated key identifiers, assessed NULL values, tested referential integrity, verified financial ranges, analyzed order statuses, calculated cancellation rate and AOV, examined monthly revenue trends, measured repeat-customer behavior, ranked customers and products, analyzed payments, and explored geographic information.

The analysis resulted in the following key metrics:

KPI	Result
Total Orders	99,441
Unique Customers	96,096
Repeat Customers	2,997
Repeat Customer Rate	3.12%
Total Item Revenue	13,591,643.70
Total Freight Value	2,251,909.54
Average Order Value	137.75
Cancellation Rate	0.63%
Order Items	112,650
Payment Records	103,886
Missing Product References	1,604
Unique Missing Product IDs	611

Overall, this section demonstrates the practical application of SQL Data Validation, Data Quality Analysis, Relational Integrity Testing, Exploratory Data Analysis, KPI Development, Customer Analytics, Revenue Analytics, Product Analytics, Payment Analytics, Time-Series Analysis, Window Functions, and Business Intelligence concepts on a large relational e-commerce dataset.

9. Advanced SQL Techniques and Analysis

The project progressed beyond basic SQL queries and implemented a range of intermediate and advanced SQL techniques to perform multi-table analysis, customer segmentation, ranking, time-series analysis, and business KPI calculations. These techniques were applied directly to the e-commerce database to answer practical business questions and demonstrate real-world SQL analytics capabilities.

9.1 SQL Joins

Joins were extensively used to combine information stored across the relational tables.

1. INNER JOIN

INNER JOIN was used when only matching records between related tables were required.

Example use cases:

- Connecting orders with customers
- Connecting order items with products
- Combining orders with transaction-level information

```
SELECT
  o.order_id,
  c.customer_unique_id,
  o.order_status
FROM orders o
INNER JOIN customers c
  ON o.customer_id = c.customer_id;
```

2. LEFT JOIN

LEFT JOIN was particularly important for data-quality analysis, because it allowed records from the primary table to be retained even when a corresponding record was missing.

Example:

```
SELECT
  oi.order_id,
  oi.product_id,
  p.product_category_name
FROM order_items oi
LEFT JOIN products p
  ON oi.product_id = p.product_id;
```

This technique was used to identify the 1,604 order-item records associated with missing product references.

9.2 Aggregate Functions

Aggregate functions were used extensively to convert transaction-level records into meaningful business metrics.

SUM()

Used for:

- Total revenue
- Total freight
- Customer revenue
- Product revenue
- Monthly revenue

```
SELECT SUM(price) AS total_revenue FROM order_items;
```

AVG()

Used to calculate:

- Average item price
- Average payment value
- Average order-related metrics

```
SELECT AVG(price) AS average_item_price FROM order_items;
```

COUNT()

Used to calculate:

- Number of orders
- Number of transactions
- Number of products
- Number of records

```
SELECT COUNT(*) AS total_orders FROM orders;
```

COUNT(DISTINCT)

Used when unique entities needed to be counted without duplication.

Examples:

- Unique customers
- Unique orders
- Unique products

```
SELECT COUNT(DISTINCT customer_unique_id) AS unique_customers FROM customers;
```

MIN() and MAX()

Used for financial and data-range validation.

```
SELECT  
  MIN(price) AS minimum_price,  
  MAX(price) AS maximum_price
```

FROM order_items;

These functions helped validate the numerical ranges before performing business analysis.

9.3 Conditional Logic Using CASE

The CASE expression was used to implement business rules and classify records into meaningful groups.

General structure:

CASE

 WHEN condition THEN result

 WHEN condition THEN result

 ELSE result

END

For example, customers were classified according to their purchasing frequency:

CASE

 WHEN order_count = 1 THEN 'One-Time Customer'

 ELSE 'Repeat Customer'

END AS customer_segment

This technique supported customer segmentation and conditional business classification.

9.4 Common Table Expressions (CTEs)

Common Table Expressions (CTEs) were used to break complex queries into smaller and more understandable logical steps.

General structure:

```
WITH customer_orders AS (  
    SELECT  
        customer_unique_id,  
        COUNT(DISTINCT order_id) AS order_count  
    FROM ...  
    GROUP BY customer_unique_id  
)  
SELECT *  
FROM customer_orders;
```

CTEs were useful for:

- Customer-level aggregation
- Repeat-customer identification

- Multi-step calculations
- Ranking preparation
- Revenue analysis
- Improving query readability

Using CTEs also made complex analytical SQL easier to debug and maintain.

9.5 Window Functions

Window functions were used to perform calculations across related rows without collapsing the result into a single aggregated row.

The project used:

- RANK()
- ROW_NUMBER()
- LAG()
- SUM() OVER()

RANK()

Used to rank customers or products according to revenue.

```
RANK() OVER (  
    ORDER BY revenue DESC  
) AS revenue_rank
```

This enabled Top-N customer and product analysis.

ROW_NUMBER()

Used to assign a unique sequential position to records within an ordered result.

```
ROW_NUMBER() OVER (  
    ORDER BY revenue DESC  
) AS row_number
```

This can be used when a unique ranking position is required even when multiple records have the same revenue.

LAG()

Used for time-series comparisons by accessing the previous row's value.

```
LAG(revenue) OVER (  
    ORDER BY month  
) AS previous_month_revenue
```

This was used to calculate Month-over-Month revenue growth.

9.6 Time-Series Analysis

Time-based analysis was performed using the order purchase timestamp.

DATE_FORMAT() was used to group transactions into monthly periods.

Example:

```
DATE_FORMAT(  
    order_purchase_timestamp,  
    '%Y-%m'  
) AS month
```

Monthly aggregation was then used to analyze revenue trends between:

September 2016 and September 2018.

This enabled analysis of:

- Monthly order volume
 - Monthly revenue
 - Revenue growth
 - Revenue fluctuations
 - Seasonal/time-based patterns
-

9.7 Running Totals

Running revenue was calculated using a windowed SUM().

Example:

```
SUM(revenue) OVER (  
    ORDER BY month  
) AS cumulative_revenue
```

This calculation provides the cumulative revenue generated over time.

The running-total analysis showed cumulative item revenue reaching: 13,591,643.70 by the end of the analyzed period.

Running totals are useful for understanding the progression of business revenue rather than looking at each month independently.

9.8 Ranking Analysis

Ranking was performed using SQL window functions.

Example:

```
RANK() OVER (  
    ORDER BY revenue DESC  
) AS revenue_rank
```

This technique was used to identify:

- Highest-revenue customers
- Highest-revenue products
- High-contribution entities
- Top-N records

Ranking allowed transaction-level data to be converted into meaningful business-level comparisons.

10. Advanced Analysis Examples

10.1 Top Customers by Revenue

Customer revenue was calculated by combining customer, order, and order-item information.

The analysis used:

- JOIN
- GROUP BY
- SUM()
- COUNT(DISTINCT)
- RANK()

The resulting dataset ranked customers according to their total item revenue.

Additional metrics such as:

- Number of orders
- Total revenue
- Average order value

could be analyzed alongside the ranking to provide a broader view of customer contribution.

10.2 Top Products by Revenue

Product-level performance was calculated by connecting order_items with products.

Revenue was calculated using:

SUM(price)

Products were then ranked using:

```
RANK() OVER (  
  ORDER BY revenue DESC  
)
```

This analysis identified products generating significant transaction revenue and demonstrated the use of JOIN, GROUP BY, aggregation, and window functions.

10.3 Monthly Revenue Analysis

Monthly revenue was calculated by extracting the year-month value from the order purchase timestamp.

```
DATE_FORMAT(  
  order_purchase_timestamp,  
  '%Y-%m'  
)
```

Revenue was then aggregated using: SUM(price)

The analysis covered the period from: 2016-09 to 2018-09 and produced a cumulative item revenue of: 13,591,643.70

10.4 Month-over-Month Revenue Growth

Month-over-Month revenue growth was calculated using the LAG() window function.

First, the previous month's revenue was retrieved:

```
LAG(revenue) OVER (
  ORDER BY month
) AS previous_month_revenue
```

The growth percentage was then calculated as:

$$\frac{(\text{Current Revenue} - \text{Previous Revenue})}{\text{Previous Revenue}} \times 100$$

Examples identified during the analysis included:

Month	MoM Growth
2017-11	+52.10%
2017-12	-26.36%
2018-01	+27.71%
2018-03	+16.47%
2018-06	-13.19%

This demonstrated practical use of LAG(), calculated expressions, and time-series analysis.

10.5 Running Revenue Analysis

A cumulative revenue metric was calculated using a windowed aggregation:

```
SUM(revenue) OVER (
  ORDER BY month
) AS cumulative_revenue
```

This allowed each month to be viewed in the context of all revenue accumulated up to that point.

The cumulative revenue reached: 13,591,643.70 at the end of the analyzed period.

10.6 Customer Segmentation

Customer segmentation was performed using order frequency.

The logic was:

CASE

```
    WHEN order_count = 1  
      THEN 'One-Time Customer'
```

```
    WHEN order_count > 1  
      THEN 'Repeat Customer'
```

```
END AS customer_segment
```

The resulting analysis identified:

Segment	Customers
One-Time Customers	93,099
Repeat Customers	2,997
Total Unique Customers	96,096

The resulting repeat-customer rate was: 3.12%

This analysis demonstrates the application of CTEs, aggregation, COUNT(DISTINCT), CASE statements, and customer-level behavioral analysis.

10.7 Revenue Contribution Analysis

Revenue contribution was used to understand how much of the total revenue was generated by an individual customer or product.

The general calculation is:

Revenue Contribution % = Individual Revenue / Total Revenue × 100

This type of analysis helps identify entities with relatively high or low contribution to the overall revenue base.

10.8 Top-N Analysis

Top-N analysis was implemented using window functions and ranking techniques.

For example:

```
RANK() OVER (  
    ORDER BY revenue DESC  
) AS revenue_rank
```

The ranking can then be filtered to identify the required number of highest-ranking entities.

Typical applications include:

- Top 5 customers
- Top 10 customers
- Top 10 products
- Top revenue-generating categories

This approach is more flexible than simply using LIMIT because window functions preserve ranking information and can also support partitioned analysis.

10.9 Advanced SQL Techniques Summary

The project incorporated the following SQL concepts:

Category	Techniques Used
Joins	INNER JOIN, LEFT JOIN
Aggregation	SUM(), AVG(), COUNT(), COUNT(DISTINCT), MIN(), MAX()
Conditional Logic	CASE, WHEN, THEN, ELSE, END
Query Structuring	CTEs / WITH
Window Functions	RANK(), ROW_NUMBER(), LAG()
Time-Series	DATE_FORMAT()
Running Totals	SUM() OVER()
Ranking	RANK() OVER()
Customer Analytics	Segmentation, repeat-purchase analysis
Revenue Analytics	Revenue contribution, AOV, monthly revenue
Trend Analysis	MoM growth and cumulative revenue

Overall, these techniques transformed the project from a simple database-querying exercise into a practical SQL analytics project involving relational data modeling, business KPI development, customer analytics, financial analysis, time-series analysis, ranking, and advanced analytical SQL.

11. Business Insights

The SQL analysis generated several business-oriented insights from customer, order, product, payment, and transaction data.

11.1 Customer Insight

The customer analysis identified 96,096 unique customers, of which 2,997 were repeat customers and 93,099 were one-time customers. The calculated repeat-customer rate was approximately 3.12%.

This provides a basis for further investigation of customer retention, repeat-purchase behavior, customer engagement, and loyalty patterns.

11.2 Sales Insight

Monthly revenue analysis showed substantial variation in sales activity across the observed period from September 2016 to September 2018.

The analysis included monthly revenue, Month-over-Month growth, and cumulative revenue, enabling identification of periods with increases and decreases in transactional sales.

The analysis recorded total item revenue of 13,591,643.70 across the analyzed transactions.

11.3 Operational Insight

Order-status analysis showed that 96,478 of 99,441 orders were recorded as delivered.

The remaining orders were distributed across statuses including shipped, canceled, unavailable, invoiced, processing, created, and approved.

This demonstrates how order-status data can be used to examine the overall order fulfillment lifecycle and identify orders that did not reach the delivered state.

11.4 Data Quality Insight

The referential-integrity analysis identified 1,604 order-item records associated with product IDs that were not present in the imported product table.

These records represented 611 unique missing product IDs, with affected transactions accounting for approximately:

- 181,469.28 in item revenue
- 28,196.81 in freight value

This finding demonstrates the importance of performing data-quality and referential-integrity validation before product-level reporting.

12. Project Challenges and Solutions

During project development, several technical and data-related challenges were encountered. These challenges provided practical experience in database management, data ingestion, troubleshooting, and data-quality analysis.

12.1 Challenge 1 — Large CSV Imports

Some of the source CSV files were relatively large, making GUI-based import through MySQL Workbench inconvenient and time-consuming.

Solution:

MySQL Shell's `util.importTable()` functionality and supporting Python import scripts were used to improve the data-ingestion process.

This provided a more efficient approach for importing large CSV datasets into MySQL.

12.2 Challenge 2 — LOAD DATA LOCAL INFILE Restrictions

The LOAD DATA LOCAL INFILE approach encountered local-file access restrictions in the MySQL environment.

Solution:

MySQL Shell import functionality was used as an alternative, with the source files prepared in an accessible local directory.

This allowed the remaining datasets to be imported successfully without relying on the restricted import method.

12.3 Challenge 3 — Referential Integrity

Some order-item records contained `product_id` values that did not have corresponding records in the products table.

Solution:

Orphan-record analysis was performed using SQL LEFT JOIN logic.

Instead of deleting these transactions, the affected records were retained and their financial impact was quantified.

12.4 Challenge 4 — Missing Delivery Dates

Several order records contained NULL values in delivery-related timestamp columns.

Solution:

NULL values were analyzed in relation to `order_status` rather than automatically treating every missing timestamp as an error.

This allowed missing dates to be interpreted in the context of the order lifecycle.

13. SQL File Documentation

The project SQL code was organized into separate files according to functionality.

13.1 01_database_schema.sql

Purpose: Creates the ecommerce_analytics database and defines the relational tables required for the project.

The file contains table definitions, column structures, data types, and key relationships used by the analytical database.

13.2 02_data_quality_audit.sql

Purpose: Performs systematic data-quality validation.

The file contains checks for:

- Duplicate identifiers
 - NULL values
 - Referential integrity
 - Orphan records
 - Financial-value ranges
 - Order-status distribution
 - Missing product references
-

13.3 03_business_analysis.sql

Purpose: Contains SQL queries for business-oriented analysis.

The analysis covers:

- Customer analysis
 - Sales and revenue analysis
 - Order-status analysis
 - Cancellation analysis
 - Monthly revenue
 - Customer segmentation
 - Product analysis
 - Payment analysis
 - Geographic analysis
 - Top-customer analysis
-

13.4 04_advanced_sql.sql

Purpose: Demonstrates advanced SQL analytical techniques.

The file includes:

- Common Table Expressions (CTEs)
- Window functions
- RANK()
- ROW_NUMBER()
- LAG()
- Running totals
- Customer segmentation
- Revenue contribution
- Top-N analysis
- Time-series analysis

This separation makes the project easier to understand, maintain, and demonstrate during technical interviews.

14. Project Limitations

The following limitations apply to the current version of the project:

1. The project primarily focuses on SQL-based data analysis.
 2. A production-grade automated ETL pipeline has not yet been implemented.
 3. The database currently runs in a local MySQL environment rather than a cloud-based platform.
 4. Automated scheduled reporting has not been implemented.
 5. The source dataset contains some referential inconsistencies, particularly missing product references.
 6. The analysis is based on a historical e-commerce dataset rather than a live transactional system.
 7. The calculated revenue metrics represent transactional item sales values and should not be interpreted as business profit or net income.
 8. The current project does not include advanced predictive modeling or machine-learning-based forecasting.
-

15. Future Enhancements

The current project provides a foundation that can be extended toward Business Intelligence and Data Engineering workflows.

15.1 Phase 2 — Business Intelligence

Potential enhancements include:

- Power BI dashboard
 - KPI cards
 - Revenue trend dashboards
 - Customer segmentation dashboard
 - Product-performance dashboard
 - Geographic performance analysis
 - Interactive filtering and drill-downs
-

15.2 Phase 3 — Data Engineering

The project can be extended into a more complete data-engineering pipeline through:

- Automated ETL pipeline
 - Python-based ingestion
 - Data warehouse implementation
 - Star-schema modeling
 - Fact and dimension tables
 - Incremental data loading
 - Automated data-quality validation
 - Pipeline monitoring and logging
-

15.3 Phase 4 — Cloud Implementation

The local MySQL architecture could be migrated or extended to cloud infrastructure using platforms such as:

- AWS
- Microsoft Azure
- Google Cloud Platform (GCP)

Potential services could include cloud databases, object storage, ETL/orchestration services, and analytical data warehouses.

15.4 Phase 5 — Advanced Analytics

Future analytical extensions could include:

- RFM customer segmentation
- Customer Lifetime Value (CLV)
- Cohort analysis
- Customer churn analysis
- Product affinity analysis
- Market-basket analysis
- Revenue forecasting
- Predictive analytics

These enhancements would extend the project from descriptive SQL analytics toward more advanced Business Intelligence, Data Engineering, and Data Analytics workflows.

16. Conclusion

This project provided practical experience in transforming raw e-commerce datasets into a structured analytical database using MySQL and SQL.

The workflow covered the major stages of an analytical database project, including database creation, relational table design, CSV data ingestion, data validation, referential-integrity analysis, financial validation, business analysis, customer segmentation, revenue analysis, time-series analysis, and advanced SQL querying.

The project worked with a large relational dataset containing 99,441 orders, 112,650 order-item records, 103,886 payment records, and 96,096 unique customers.

Advanced SQL techniques including JOINS, aggregations, CASE expressions, CTEs, window functions, ranking, LAG(), running totals, and time-based analysis were applied to convert raw transactional data into meaningful business metrics.

The project also demonstrated practical troubleshooting during data ingestion and documented limitations in the source data rather than ignoring them. This provided hands-on experience with both the technical and analytical aspects of working with real-world datasets.

The resulting SQL project establishes a foundation for future development into Business Intelligence dashboards, automated ETL pipelines, data warehouses, cloud-based data platforms, and advanced customer analytics.

17. GitHub Repository

Project Repository

Ecommerce Customer Sales Intelligence

GitHub Repository:

https://github.com/DiveshSonawane/Ecommerce_Customer_Sales_Intelligence

The repository contains the project source files, including:

- SQL database schema
- Data-quality audit queries
- Business-analysis queries
- Advanced SQL queries
- Python data-import scripts
- Project configuration files
- Dataset files, where applicable

The repository provides a reproducible reference for the SQL analysis workflow and allows the project structure and implementation to be reviewed independently.
